Comparative Analysis of Graph-based Extractive Text Summarization

1st Manasi Waghe
Computer Engineering
Pune Institute of Computer Technology
Pune, India
manasi.waghe12@gmail.com

2nd Danish Chandargi
Computer Engineering
Pune Institute of Computer Technology
Pune, India
danishchandargi@gmail.com

3rd Mohammad Aamir Rayyan
Computer Engineering
Pune Institute of Computer Technology
Pune, India
aamirray19@gmail.com

4th Sheetal Sonawane
Computer Engineering
Pune Institute of Computer Technology
Pune, India
sssonawane@pict.edu

***Abstract*—Text summarization for low-resource languages such as Hindi presents inherent challenges owing to the limited availability of high-quality linguistic resources and curated datasets. In this study, we examine and compare three distinct methodologies for extractive summarization of Hindi texts: FastText-based semantic similarity, WordNet-based lexical similarity, and the PageRank algorithm for sentence ranking. The summarization pipeline involves segmenting the input text into individual sentences, computing pairwise sentence similarity by employing either distributed word embeddings (FastText) or lexical-semantic relationships (WordNet), and subsequently constructing a weighted graph wherein nodes represent sentences and edges denote similarity scores. The PageRank algorithm is then applied to this graph to identify and extract the most salient and informative sentences. We evaluate and contrast the quality of summaries generated by each approach through a detailed analysis of their output scores, illustrating how each technique captures different dimensions of relevance and coherence. This comparative analysis offers insights into the relative effectiveness of semantic, lexical, and structural techniques for Hindi text summarization and highlights the potential advantages of integrating these approaches for improved summarization performance.**

***Index Terms*—Hindi, Text Summarization, FastText, WordNet, PageRank, Sentence Similarity, Extractive Summarization, Natural Language Processing**

I. Introduction

With the rapid proliferation of digital content in regional languages, automatic text summarization has emerged as a pivotal task within the domain of Natural Language Processing (NLP). Summarization entails generating a condensed yet semantically rich representation of a document while preserving its essential information. Although substantial research has been conducted for English and other resource-rich languages, Hindi—a morphologically rich and widely spoken Indian language—continues to lack robust summarization frameworks and linguistic resources.

Hindi text summarization encounters distinct challenges arising from the scarcity of large-scale annotated corpora, the limited availability of high-quality linguistic tools, and the intricacy of sentence structures. Extractive summarization, which selects salient sentences directly from the source text, presents a pragmatic solution for such low-resource contexts. This work focuses on examining and comparing three methodologies for extractive summarization in Hindi: FastText embeddings, WordNet-based lexical similarity, and the PageRank algorithm.

FastText, a distributed word embedding model developed by Facebook AI Research, captures both semantic and subword-level representations, rendering it well-suited for morphologically complex languages such as Hindi. In contrast, WordNet serves as a lexical database that organizes words into sets of cognitive synonyms (synsets), facilitating similarity computation through lexical-semantic relationships. The PageRank algorithm, originally designed for ranking web pages, is adapted here to rank sentences according to their centrality within a similarity graph derived from the document.

In this paper, we construct a modular summarization pipeline in which each of these techniques is applied independently to assess their individual effectiveness. The pipeline consists of sentence segmentation, similarity matrix computation via either FastText or WordNet, weighted graph construction, and sentence ranking using PageRank. We then evaluate and compare the summaries produced by each method to discern their respective strengths and limitations.

By systematically analyzing these techniques, this study offers insights into the comparative performance of semantic, lexical, and graph-based approaches for Hindi extractive summarization. The findings underscore their individual contributions and highlight the potential for developing hybrid architectures to advance NLP capabilities for low-resource languages.

II. Literature Survey

Conventional extractive text summarization, particularly in resource-rich languages, tends to depend on statistical and

token-overlap-based approaches in order to find key sentences. The core of such approaches lies in mechanisms such as overlapping token counting between sentences or vector-based similarity measures like Cosine and Jaccard similarity. Their success lies in their direct correspondence with n-gram overlap-based evaluation metrics like ROUGE, which inherently bias towards summaries with higher lexical similarity to a reference summary. These approaches, such as ROUGE-N (n-gram overlap), ROUGE-L (longest common subsequence), and ROUGE-S (skip-bigram), have been deep-rooted as a benchmark for automatic summarization task evaluation [4]. From the comparative study demonstrated in this manuscript, it is evident that conventional cosine similarity always proves to be among the top performers in ROUGE-based evaluations, achieving maximum F1-scores for varying dataset sizes, hence suggesting its validity for such evaluations. These lexical-based approaches are typically integrated with graph-based approaches like TextRank [1], where the document is modeled as a graph and sentence significance as inferred from the graph structure.

The method presented in this paper extends current graph-based methodology by conducting comparative analysis of more semantically informative similarity measures, which is particularly crucial for morphologically dense, low-resource languages like Hindi. The main method imagines a Hindi text as a complete, undirected graph, where sentences are nodes and weights on edges are a similarity measure. Then, the PageRank algorithm ranks sentences for relevance and centrality in the graph [2], [3]. Specifically, this work compares and assesses two novel similarity measures to baselines: WordNet-based lexical similarity and FastText-based semantic similarity. To move away from lexical similarity, the work explores dense vector representations of words, a paradigm that has risen to prominence through models like Word2Vec, which learn embeddings from large text corpora [3]. Based on this, the work employs pre-trained 300-dimensional FastText embeddings, retaining semantic and sub-word information, a necessary component for dealing with the morphological richness and out-of-vocabulary words common in Hindi. Meanwhile, in an effort to exploit curated linguistic knowledge, the work employs IndoWordNet, defining similarity as the intersection of synonym sets (synsets) to pick out semantically relevant sentences, even in the absence of direct lexical overlap. This comprehensive comparison study offers a rich understanding of the efficacies of lexical, semantic, and structural methods, ultimately concluding that, while token-based similarity is breathtakingly effective in ROUGE-centric evaluations, semantic methods increasingly promise, particularly as datasets grow.

III. Methodology

This section delineates the algorithmic framework employed for graph-based Automatic Text Summarization (ATS) of Hindi texts. It specifies the datasets leveraged and introduces proposed variations in similarity measures for evaluating inter-sentence relationships. The overall architecture of the proposed system is depicted in Figure 1, encapsulating the primary stages: preprocessing, graph construction, sentence ranking via PageRank, and final summary generation [1], [7], [13], [14].

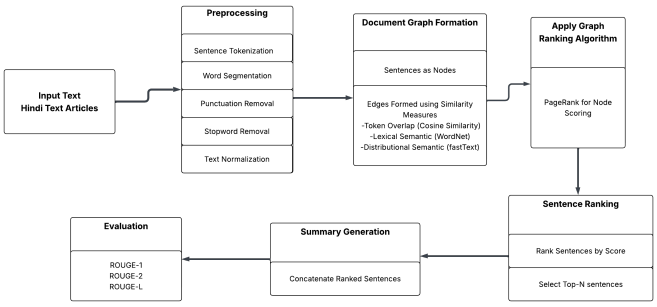


Fig. 1. System Architecture

A. Graph Based Algorithm

Hindi text documents are processed individually, with a fully connected graph constructed for each document. In this representation, each sentence corresponds to a vertex, and edges are weighted according to computed similarity measures that quantify sentence-level relationships. The methodology follows a process inspired by Barrios et al. [7], facilitating the application and evaluation of multiple sentence similarity techniques [2], [6], [14].

1) Pre-processing: The input Hindi article is segmented into sentences. Each sentence undergoes cleaning (removal of punctuation, digits, and non-Devanagari characters). However, the final output summary retains punctuation. After sentence segmentation, words are extracted using the Indic NLP Library tokenizer [8]. Preprocessing steps include:

- Lowercasing
- Unicode normalization
- Stopword removal (using a Hindi stopword list [11])
- Optional lemmatization (for word-level similarity [12])

2) Graph Formation: An undirected graph is created where each node represents a sentence from the article. Edges are formed between every pair of nodes (fully connected), and edge weights represent sentence similarity using the chosen measure [9], [13]. Let

$$G = (V, E) \tag{1}$$

denote the graph, where:

- V: Set of all sentences
- $E(i, j)$: Similarity score between sentences S_i and S_j

3) Sentence Ranking: We determine the significance of each sentence by applying the PageRank algorithm, which evaluates how each sentence is connected to others within the similarity graph. PageRank scores each sentence using:

$$PR(S_i) = (1 - d) + d \sum_{S_j \in \text{In}(S_i)} \frac{PR(S_j)}{\text{outdeg}(S_j)} \tag{2}$$

where $d = 0.85$ is the damping factor. The graph is internally converted to a directed one for compatibility [9], [13].

4) Sentence Selection: Sentences are ranked by their PageRank score in descending order. The top-ranked sentences are selected until a word limit threshold is reached (e.g., 100–120 words), forming the extractive summary [1], [7], [14].

B. Hindi Dataset

The methodology utilizes three publicly available datasets to support both the extractive and abstractive summarization tasks, as well as preprocessing:

1) *Hindi Text Short and Large Summarization Corpus:* Sourced from Kaggle, this corpus consists of approximately 180k to 330k news articles, each paired with a short summary (headline-style) and a longer highlight in Devanagari script [5]. The average length of short summaries is 80–100 words. This dataset is used for:
   - Graph-based extractive summarization (comparing system-generated summaries to human-written ones using ROUGE metrics) [4].
   - Fine-tuning transformer-based abstractive models, with full text as input and the human-written short summary as the target [15], [16].
2) *Hindi Stopwords and Sentiment Lexicon:* A curated list of Hindi stopwords and sentiment lexicons, sourced from Kaggle [11]. It is employed in the preprocessing stage to remove high-frequency, non-informative tokens, thereby improving signal clarity in both TF-IDF and graph-based embeddings [6], [17].

C. Taxonomy of Similarity Measures

1) Word-Level Similarity Measures:

a) Common Tokens: This measure calculates the number of common tokens between two sentences S_i and S_j, as shown in (3). It was originally used in the TextRank algorithm [9], [13].

$$\text{Sim}_{\text{common}}(S_i, S_j) = \frac{|\{\text{tokens in } S_i\} \cap \{\text{tokens in } S_j\}|}{\log(|S_i|) + \log(|S_j|)} \quad (3)$$

b) Cosine Metric (Ochiai Coefficient): Given two sentences A and B, the Ochiai coefficient K is defined as:

$$K(A, B) = \frac{|A \cap B|}{\sqrt{|A| \cdot |B|}} \quad (4)$$

If either A or B is an empty set, then $K = 0$. If both are empty, then $K = 1$.

c) Jaccard Metric: This metric assumes that sentences A and B are independent sets of words. The Jaccard similarity is defined in (5) [2], [6].

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (5)$$

If both A and B are empty sets, then $J = 1$.

2) fastText-Based Semantic Similarity: To capture semantic relationships beyond lexical overlap, we employ pre-trained fastText word embeddings. These are 300-dimensional vectors trained using the Continuous Bag-of-Words (CBOW) model with character n-grams (length 5) on Common Crawl and Wikipedia data. Notably, fastText supports Out-Of-Vocabulary (OOV) words by constructing embeddings from subword components, making it particularly effective for low-resource languages.

Each sentence is represented by averaging the fastText vectors of its constituent words, resulting in a dense, fixed-length semantic vector [3], [10].

We use the following four vector-based similarity measures:

- *Cosine Similarity:* Measures angular similarity between sentence vectors.
- *Euclidean Similarity:* Transforms Euclidean distance into similarity using an exponential decay function.
- *Manhattan (City Block) Similarity:* Uses the sum of absolute differences and converts it into similarity.
- *Jaccard Similarity:* Applied to binarized sentence embeddings to indicate overlap in active semantic dimensions.

These similarity scores form the edge weights in the sentence graph used for extractive summarization.

3) WordNet-based Semantic Similarity: To capture deeper semantic relationships between Hindi sentences, we utilize IndoWordNet, a lexical ontology for Indian languages [12]. The similarity between two sentences is computed by analyzing the overlap of their synsets (semantic sets of meanings) retrieved from IndoWordNet.

Each sentence is tokenized using the nltk.word_tokenize function. For every word pair across two sentences, synset intersections are counted to compute a normalized semantic similarity score, defined as:

$$\text{Similarity}(S_1, S_2) = \frac{|\text{Synsets}(S_1) \cap \text{Synsets}(S_2)|}{|S_1| \cdot |S_2|} \quad (6)$$

These pairwise similarities form a sentence similarity matrix, which is used to construct a graph where nodes represent sentences and edges are weighted by semantic similarity. The graph is then ranked using the PageRank algorithm, and the top-ranked sentences are selected for summary generation [7], [13], [14].

To ensure diversity in the output, highly similar sentences (similarity score ≥ 0.7) are filtered out during the selection phase. The generated summaries are evaluated using ROUGE-1, ROUGE-2, and ROUGE-L metrics across different dataset sizes.

IV. Results

We evaluated the performance of the proposed Hindi text summarization approaches across four dataset sizes: 30, 50, 100, and 200 documents. Each method was assessed using ROUGE-1, ROUGE-2, and ROUGE-L metrics, reporting precision, recall, and F1-scores. The consolidated results are presented in Table I, which provides a comparative performance analysis for all methods across the four dataset configurations.

TABLE I
ROUGE-1, ROUGE-2, AND ROUGE-L PRECISION, RECALL, AND F1-SCORES

Docs	Method	ROUGE-1			ROUGE-2			ROUGE-L		
		Prec	**Rec**	**F1**	**Prec**	**Rec**	**F1**	**Prec**	**Rec**	**F1**
30	Traditional (Common Tokens)	**0.242**	0.541	0.331	0.099	0.251	0.141	**0.224**	0.499	**0.306**
	Traditional (Cosine)	0.237	0.558	0.329	0.098	0.274	0.142	0.221	0.518	0.304
	Traditional (Jaccard)	0.227	0.534	0.315	0.084	0.241	0.124	0.209	0.495	0.291
	WordNet	0.231	**0.608**	**0.332**	0.104	0.324	0.155	0.211	0.552	0.295
	FastText (Cosine)	0.176	0.426	0.241	0.055	0.177	0.083	0.170	0.411	0.229
	FastText (Euclidean)	0.178	0.437	0.246	0.056	0.175	0.085	0.171	0.422	0.234
	FastText (Manhattan)	0.182	0.452	0.252	0.058	0.186	0.088	0.174	0.437	0.240
50	Traditional (Common Tokens)	0.272	0.598	0.368	0.139	0.339	0.193	0.256	0.576	0.346
	Traditional (Cosine)	**0.275**	0.615	**0.375**	**0.145**	0.372	**0.205**	**0.261**	0.594	**0.353**
	Traditional (Jaccard)	0.268	0.603	0.366	0.133	0.351	0.190	0.251	0.577	0.341
	WordNet	0.255	**0.635**	0.358	0.131	**0.370**	0.190	0.243	**0.603**	0.331
	FastText (Cosine)	0.171	0.416	0.236	0.048	0.148	0.072	0.165	0.403	0.224
	FastText (Euclidean)	0.172	0.422	0.238	0.047	0.142	0.070	0.166	0.408	0.226
	FastText (Manhattan)	0.174	0.431	0.242	0.048	0.150	0.072	0.168	0.417	0.230
100	Traditional (Common Tokens)	0.272	0.594	0.368	0.141	0.351	0.198	0.258	0.572	0.348
	Traditional (Cosine)	**0.278**	0.601	**0.375**	0.142	0.366	0.202	**0.264**	0.582	**0.354**
	Traditional (Jaccard)	0.266	0.573	0.357	0.128	0.326	0.180	0.249	0.557	0.338
	WordNet	0.271	**0.644**	0.374	**0.144**	**0.397**	**0.206**	0.251	**0.610**	0.336
	FastText (Cosine)	0.203	0.470	0.273	0.077	0.233	0.112	0.198	0.462	0.267
	FastText (Euclidean)	0.209	0.492	0.284	0.082	0.243	0.119	0.202	0.482	0.273
	FastText (Manhattan)	0.211	0.490	0.285	0.083	0.244	0.120	0.204	0.480	0.274
200	Traditional (Common Tokens)	0.272	0.594	0.368	0.141	0.351	0.198	0.258	0.572	0.348
	Traditional (Cosine)	**0.278**	0.601	**0.375**	0.142	0.366	0.202	**0.264**	0.582	**0.354**
	Traditional (Jaccard)	0.266	0.573	0.357	0.128	0.326	0.180	0.249	0.557	0.338
	WordNet	0.271	**0.644**	0.374	**0.144**	**0.397**	**0.206**	0.251	**0.610**	0.336
	FastText (Cosine)	0.240	0.537	0.321	0.116	0.334	0.168	0.233	0.521	0.307
	FastText (Euclidean)	0.244	0.546	0.327	**0.120**	**0.341**	**0.173**	0.238	0.530	0.312
	FastText (Manhattan)	0.244	0.544	0.327	0.119	0.333	0.170	0.236	0.528	0.310

Table I reveals that, across all dataset sizes, traditional extractive summarization with cosine similarity consistently attained the highest F1-scores. For instance, in the 30-document set, the cosine-based method achieved a ROUGE-1 F1-score of 0.329 and a ROUGE-L F1-score of 0.304. This trend persisted with larger datasets, where the 50-, 100-, and 200-document configurations all yielded a ROUGE-1 F1-score of 0.375. The stability of these results across scales underscores the robustness of cosine similarity in generalizing to varied input sizes.

The WordNet-based approach demonstrated notably high recall in all configurations, outperforming other methods in recall-oriented evaluations. For example, in the 100-document set, WordNet achieved a ROUGE-1 recall of 0.644, surpassing cosine similarity (0.601). Furthermore, it obtained the highest ROUGE-2 F1-score of 0.206 at both the 100- and 200-document levels. These results highlight WordNet's capacity to capture semantic relationships that transcend exact lexical matches, enabling the extraction of conceptually relevant sentences.

In contrast, fastText-based semantic methods underperformed in smaller datasets. For the 30-document set, their ROUGE-1 F1-scores ranged between 0.241 and 0.252. However, performance improved with dataset size. Notably, the Manhattan similarity variant achieved its best performance at 200 documents, with a ROUGE-1 F1-score of 0.327 and a ROUGE-2 F1-score of 0.170. This pattern suggests that dense semantic embeddings require larger and more diverse datasets to realize competitive performance.

A. Comparative Insights

From the evaluation presented in Table I, several key insights emerge:

- Cosine Similarity as a Consistent Baseline: Across all dataset sizes, cosine similarity achieved the highest ROUGE-1 and ROUGE-L F1-scores. Its effectiveness is attributable to its alignment with ROUGE's n-gram overlap mechanism, which inherently favors lexical similarity.
- WordNet for Enhanced Semantic Coverage: WordNet-based similarity consistently achieved the highest recall and competitive ROUGE-2 F1-scores, reflecting its capability to capture deep semantic relations—particularly advantageous for longer or semantically diverse documents.
- fastText Performance Improves with Scale: fastText-based approaches showed marked performance gains with larger datasets, especially in the Manhattan variant for the 200-document set. Nonetheless, their relatively lower ROUGE F1-scores suggest that while they provide se-

mantic robustness, they are disadvantaged by ROUGE's emphasis on surface-level token overlap.

In summary, while cosine similarity remains optimal for ROUGE-centric evaluations, WordNet offers superior recall and semantic generalization, making it well-suited for large-scale summarization tasks. fastText models exhibit scaling potential, yet their evaluation under ROUGE remains limited, underscoring the need for semantic-aware evaluation metrics in future research.

V. Conclusion

This study presents a comparative evaluation of extractive Hindi text summarization techniques employing graph-based architectures in conjunction with multiple similarity measures, namely token overlap, lexical semantic similarity via WordNet, and distributional semantic similarity using fastText embeddings. Experimental results across progressively larger datasets indicate that traditional cosine similarity consistently achieved the highest ROUGE F1-scores, establishing it as the most effective approach under ROUGE-based evaluation criteria.

The WordNet-based methodology exhibited superior recall and demonstrated increased competitiveness with larger datasets, underscoring its capacity for broader semantic coverage. Conversely, while fastText-based approaches underperformed in ROUGE evaluations due to reduced lexical overlap, they displayed marked improvement at larger scales, suggesting considerable potential for semantically enriched summarization.

Overall, the findings affirm the enduring effectiveness of token-based similarity for extractive summarization in ROUGE-oriented evaluations, while simultaneously highlighting the emerging promise of semantic similarity techniques for more contextually aware and conceptually comprehensive summarization tasks.

References

[1] Rada Mihalcea and Paul Tarau. 2004. TextRank: Bringing Order into Text. In Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing, pages 404–411, Barcelona, Spain. Association for Computational Linguistics.

[2] Barrios, F., López, F., Argerich, L., & Wachenchauzer, R. (2016). Variations of the Similarity Function of TextRank for Automated Summarization. arXiv preprint arXiv:1602.03606.

[3] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.

[4] Lin, C. Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. In Proceedings of the ACL-04 Workshop on Text Summarization Branches Out.

[5] Vishwajeet Kumar, Nishant Shukla, and Manish Shrivastava. 2019. Deep Learning Based Text Summarization Approaches on Hindi Language Dataset. In CICLing, 180–192.

[6] Jovi D'Silva and Uzzal Sharma. 2023. Impact of Similarity Measures in Graph-based Automatic Text Summarization of Konkani Texts. ACM Trans. Asian Low-Resour. Lang. Inf. Process. 22, 2, Article 51. https://doi.org/10.1145/3554943

[7] Alejandro Barrios, Federico López, Luis Marujo, and Jaime Carbonell. 2016. Variations of the Similarity Function of TextRank for Automated Summarization. In Recent Advances in Natural Language Processing, 48–55.

[8] Pawan Goyal and Kalika Bali. 2016. Indic NLP Library. Available at: https://github.com/anoopkunchukuttan/indic_nlp_library

[9] Rada Mihalcea and Paul Tarau. 2004. TextRank: Bringing Order into Texts. In EMNLP, 404–411.

[10] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. Enriching Word Vectors with Subword Information. Transactions of the Association for Computational Linguistics 5, 135–146.

[11] K. Sarkar. 2010. An Approach to Summarizing Bengali Texts. In 1st Workshop on South and Southeast Asian NLP (WSSANLP), 22–30.

[12] A. Kulkarni, R. Sharma, S. Kumar, and P. Bhattacharyya. 2010. Introducing a Lexical Resource for Marathi using WordNet: IndoWordNet. In LREC, 2520–2525.

[13] Erkan, G., & Radev, D. R. (2004). LexRank: Graph-based lexical centrality as salience in text summarization. Journal of Artificial Intelligence Research, 22, 457–479.

[14] Yasunaga, M., Zhang, R., Meelu, K., Pareek, A., Srinivasan, K., & Radev, D. (2017). Graph-based Neural Multi-Document Summarization. In Proceedings of CoNLL 2017, 452–462.

[15] Wu, W., Wei, F., Zhou, M., & Xu, W. (2021). BASS: Boosting Abstractive Summarization with Unified Semantic Graph. arXiv preprint arXiv:2105.12041.

[16] Cui, P., Hu, L., & Liu, Y. (2020). Enhancing Extractive Text Summarization with Topic-Aware Graph Neural Networks. arXiv preprint arXiv:2010.06253.

[17] Li, C., Gao, Y., & Huang, T. (2023). Knowledge-graph based multi-document text summarization for mobile edge computing. Journal of Cloud Computing, 12(1), 1–16.

[18] Zhang, J., Chen, H., & Liu, Y. (2024). Text summarization based on semantic graphs: an abstract meaning framework. Journal of Big Data, 11(1), 1–22.

[19] Singh, A., & Sharma, R. (2024). Graph-based Extractive Text Summarization Sentence Scoring (GETS). Information, 14(9), 472.

[20] Cohan, A., Dernoncourt, F., Kim, D. S., Bui, T., Kim, W., Chang, W., & Goharian, N. (2021). CGSum: Citation Graph-based Scientific Paper Summarization. arXiv preprint arXiv:2104.03057.

[21] Kryściński, W., McCann, B., Xiong, C., & Socher, R. (2022). Evaluating Factuality in Summarization with Semantic Graph Representations (FactGraph). arXiv preprint arXiv:2204.06508.